



Recipe App

Name: Niță Maria-Ștefania

Group: 30424

1. Project Overview

RecipeApp is a desktop application built with Java, that allows users to discover, create, and share cooking recipes. It is designed to act like a digital cookbook where home cooks can save their favourite dishes and review recipes posted by others.

The application uses a **Graphical User Interface (GUI)** to make it easy to use and connects to a **SQL Database** to safely store all user accounts, recipes, and reviews.

2. Key Features

User Features

- **Login & Register:** Secure account creation and login system.
- **Browse Recipes:** View a list of available recipes with images, prep time, and serving sizes.
- **Search:** Find recipes quickly by searching for a name (e.g., "Pasta") or a category (e.g., "Italian").
- **Create Recipes:** Users can add their own recipes, including ingredients, instructions, and photos.
- **Reviews:** Users can rate recipes (1-5 stars) and leave comments.
- **My Account:** A personal dashboard to manage your own recipes and reviews.

Admin Features

- **User Management:** Administrators can view all registered users and delete accounts if necessary.
- **Content Moderation:** Admins can delete any recipe that violates rules.

3. Arhitecture

The project is organized into three main layers (folders) to keep the code clean:

1. **ui (User Interface):**

- This folder contains everything the user sees.
- **Login.java:** Handles the login screen.
- **RecipeApp.java:** The main dashboard where users browse recipes.
- **MainFrame.java:** A helper class that stores colours, fonts, and custom button styles to keep the design consistent.

2. **service (Logic & Database):**

- **DataBase.java:** Handles the connection to the SQL Server.
- **RecipeService.java:** The "brain" of the app. It handles complex tasks like searching for recipes, saving a new review, or checking if a password is correct.

3. **model (Data Objects):**

- These are simple classes that represent real-world things.
- **User / Admin:** Represents a person using the app.
- **Recipe:** Represents a dish (name, ingredients, time).
- **Review:** Represents a comment and rating.

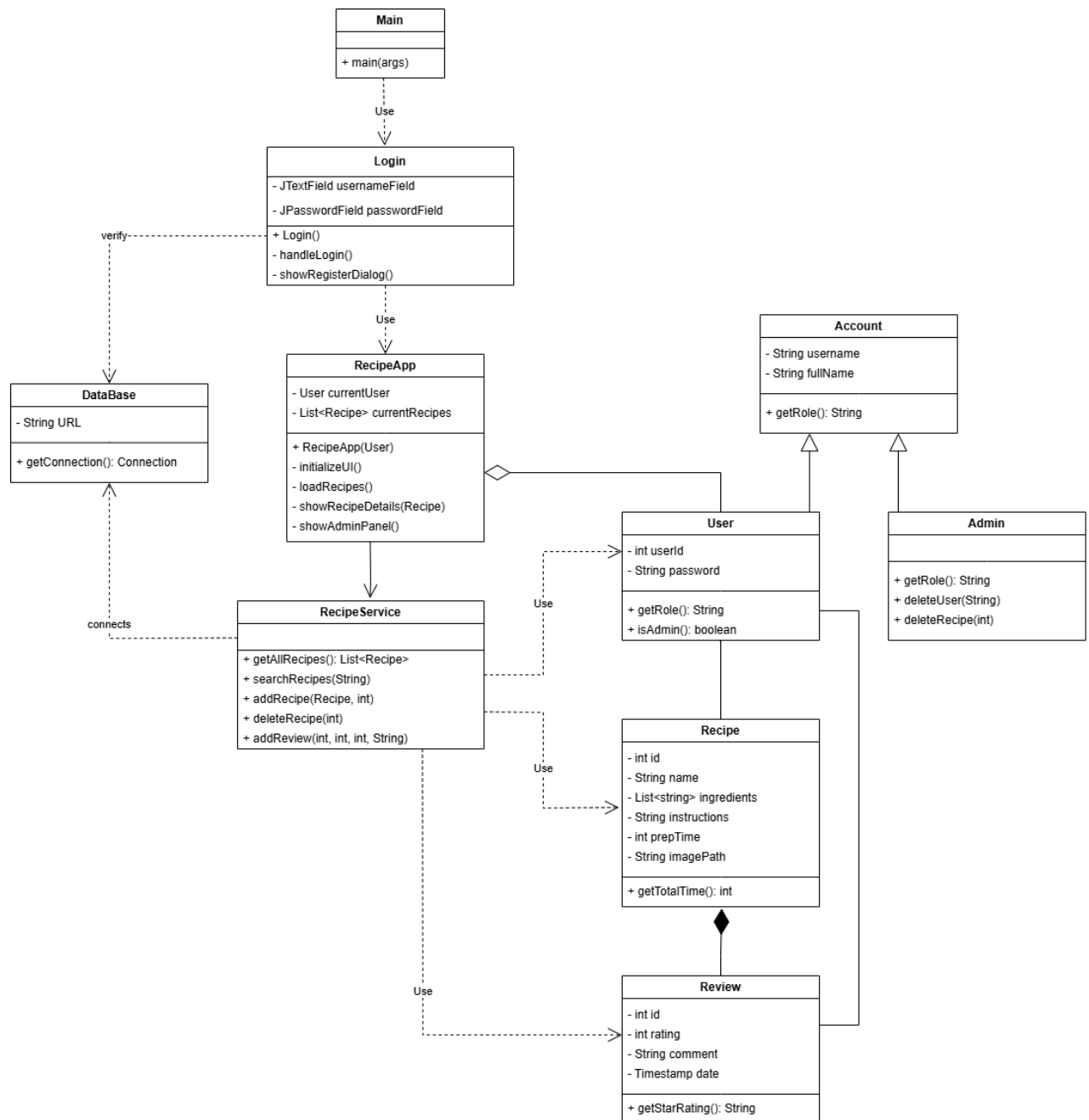
4. Database Setup

The application uses **Microsoft SQL Server**.

- **Database Name:** Project
- **Tables:** The system requires tables for Users, Recipes, Reviews, and Ingredients.

5. UML Diagram

The UML Diagrams exemplifies the relationship between classes:



Source Class	Target Class	Relationship Type	Description
Main	Login	Dependency (Use)	The Main class launches the Login window.
Login	RecipeApp	Dependency (Use)	Upon successful login, Login creates the RecipeApp.
Login	DataBase	Dependency (Verify)	Login uses DataBase to verify user credentials.
RecipeApp	User	Aggregation	RecipeApp holds the active User session
RecipeApp	RecipeService	Aggregation	RecipeApp maintains a RecipeService instance to handle logic
User	Account	Inheritance	User extends the Account abstract class.
Admin	Account	Inheritance	Admin extends the Account abstract class.
RecipeService	DataBase	Dependency (Connects)	RecipeService connects to the DataBase to fetch data.
RecipeService	User	Dependency (Use)	RecipeService uses User objects for data processing.

Source Class	Target Class	Relationship Type	Description
RecipeService	Recipe	Dependency (Use)	RecipeService manages Recipe objects.
RecipeService	Review	Dependency (Use)	RecipeService manages Review objects.
Recipe	Review	Composition	The diagram shows Recipe owns Review implying reviews cannot exist without a recipe.
User	Recipe	Association	The diagram shows a link between User and Recipe

6. OOP Principles Applied:

Encapsulation

Encapsulation means keeping data safe by hiding it inside classes and only allowing access through specific methods.

- **How it's used:** In the User and Recipe classes, all fields (like password, userId, recipeName) are marked as private. This prevents other parts of the code from changing them directly.
- **Example:** You cannot change a user's ID directly, you must use `user.getUserId()` to read it or a constructor to set it safely.

Inheritance

Inheritance allows new classes to be created based on existing ones, reusing code to avoid repetition.

- **How it's used:** We created a base class called Account that holds common data like username and fullName.
- **The Result:** Both User (for regular home cooks) and Admin (for administrators) extend Account. This means they automatically get those common fields without us having to write the same code twice.

Polymorphism

Polymorphism allows different classes to be treated as the same type but behave differently.

Overriding:

- In User.java and Admin.java, we **override** the getRole() method from the abstract Account class to return different strings ("Home Cook" vs "Administrator").
- In the UI classes (like Login.java), we **override** paintComponent(Graphics g) to draw the custom rounded backgrounds and images.

Overloading:

- In Recipe.java, we have **overloaded constructors**:
- public Recipe(int id, ...) - Used when fetching an existing recipe from the database (includes the ID).
- public Recipe(String name, ...) - Used when creating a *new* recipe (no ID yet).

Abstraction

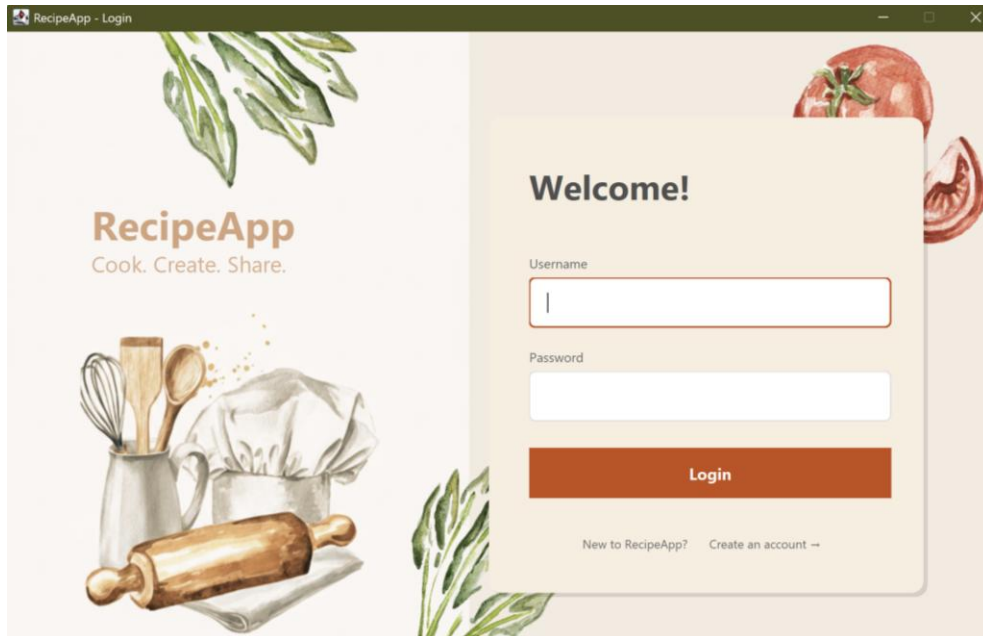
Abstraction is about hiding complex details and showing only what is necessary.

- **How it's used:**
 - **Abstract Classes:** The Account class is abstract, meaning you cannot create a generic "Account"—you must create specifically a User or an Admin.
 - **Services:** The RecipeService hides the complex SQL database code. The GUI doesn't know *how* to talk to the database; it just calls simple methods like getAllRecipes(), and the service handles the rest.

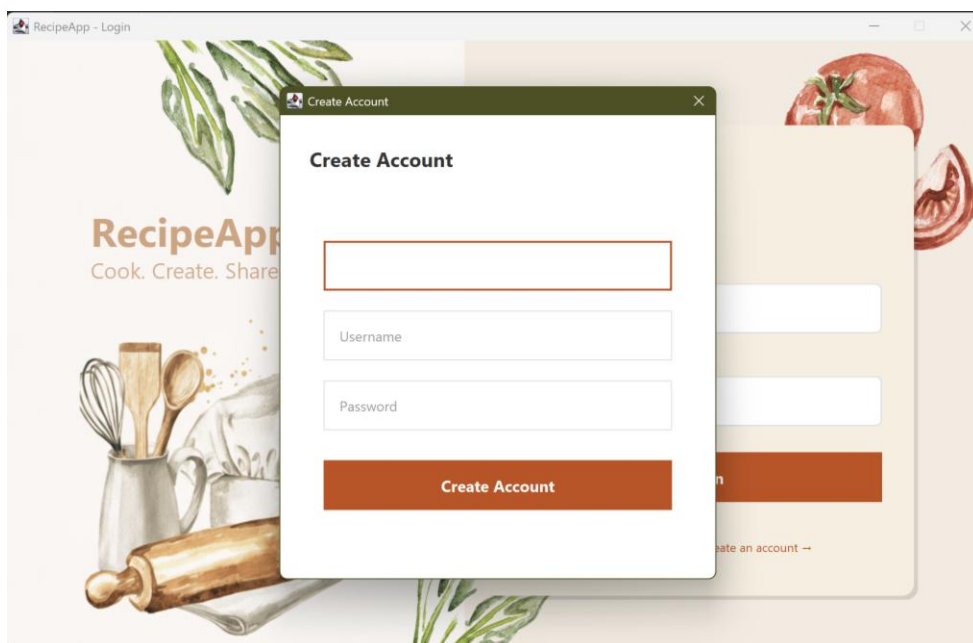
7. App function:

Login Page:

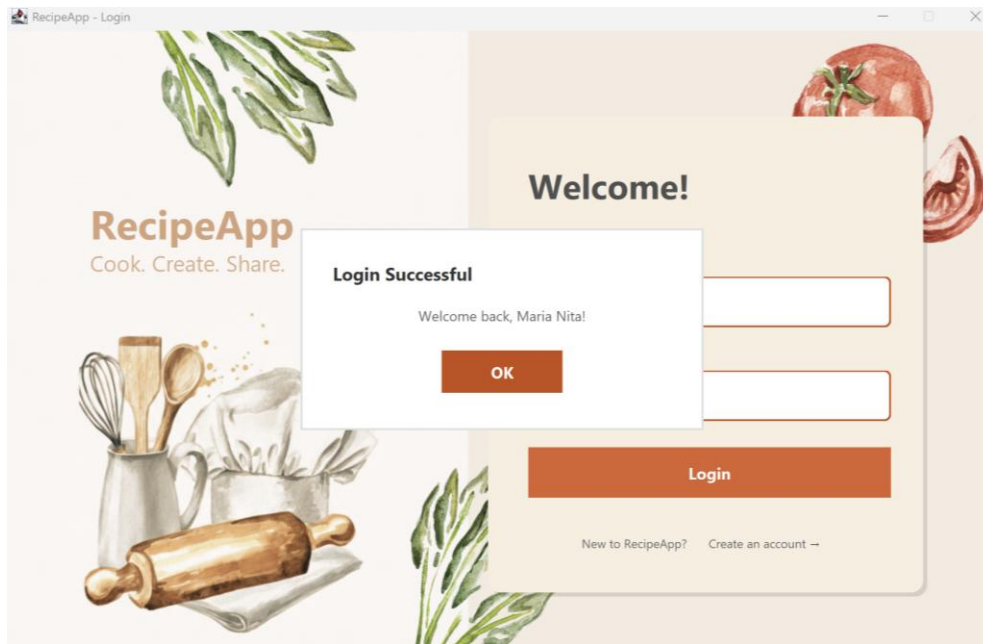
When you run the programme this the first window that opens



If you do not have an account, press Create an account, in order to create a user account.

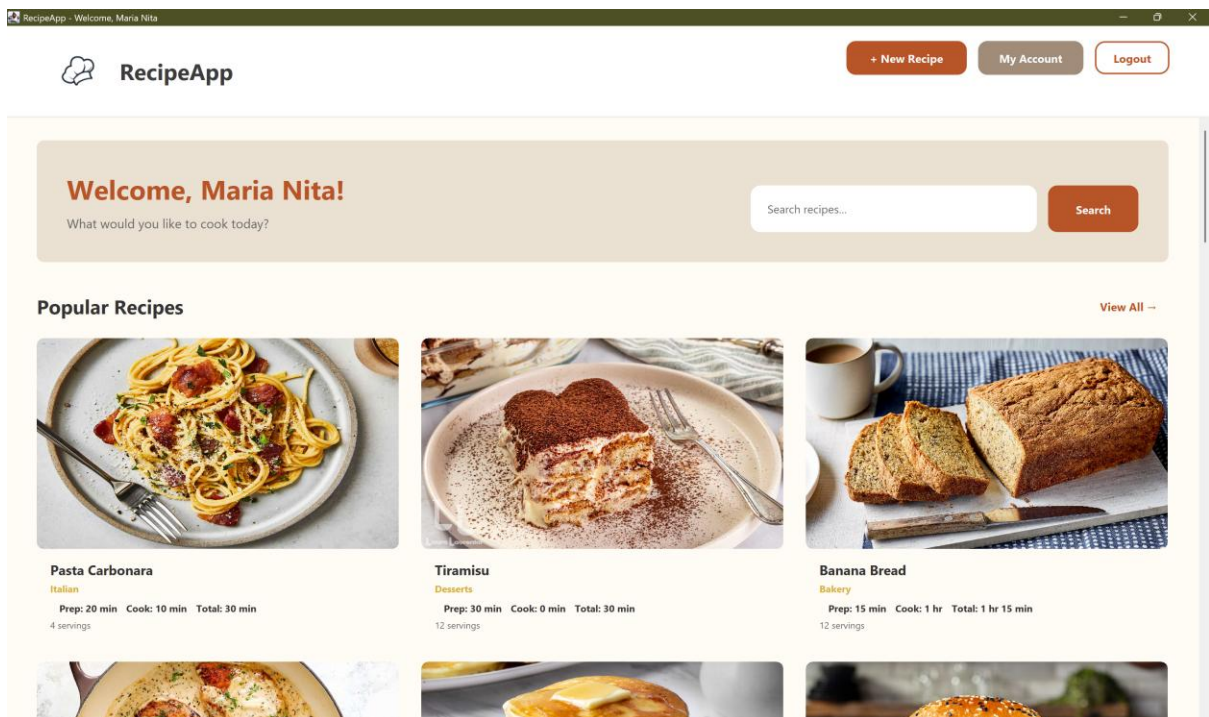


Next step: if the user either has an account or already created one, now he has to login correctly.

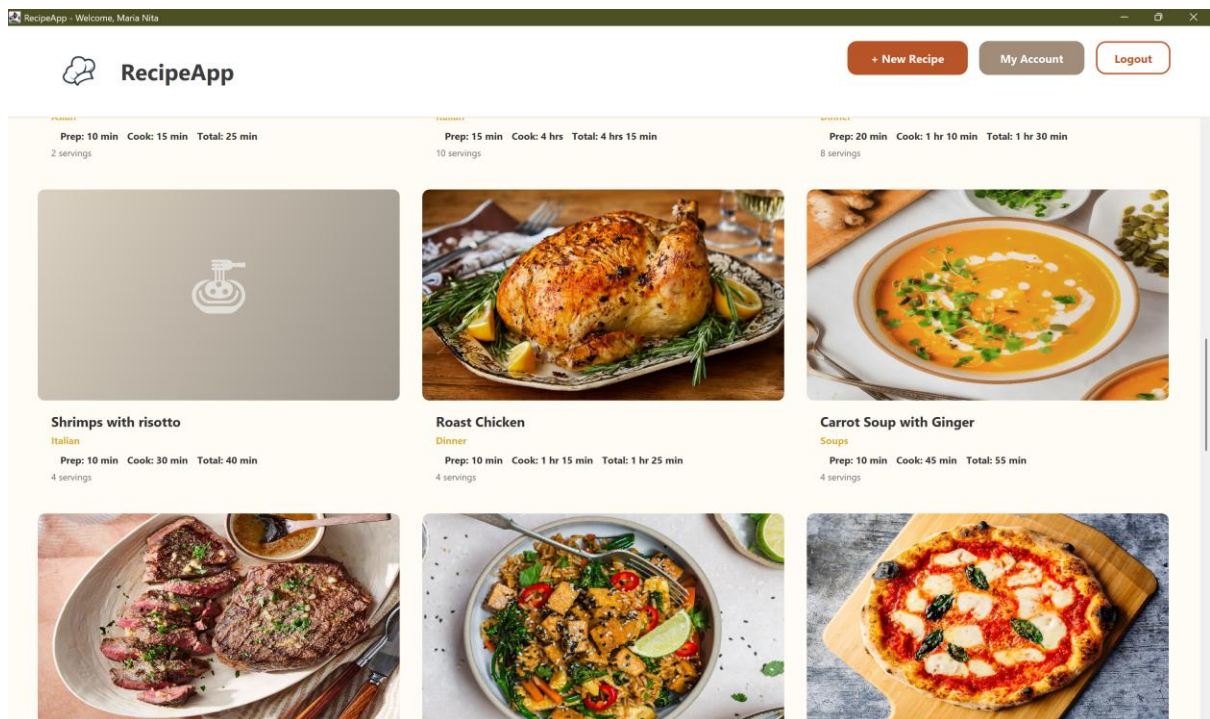


User page:

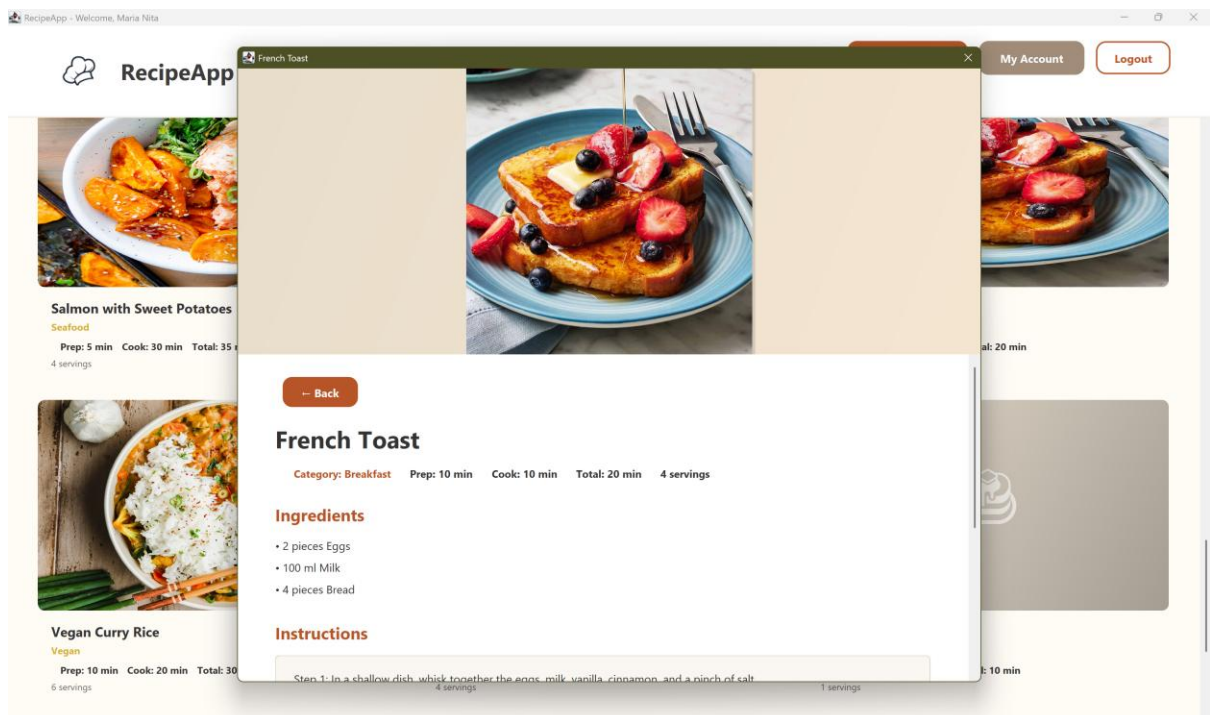
After a successful login:

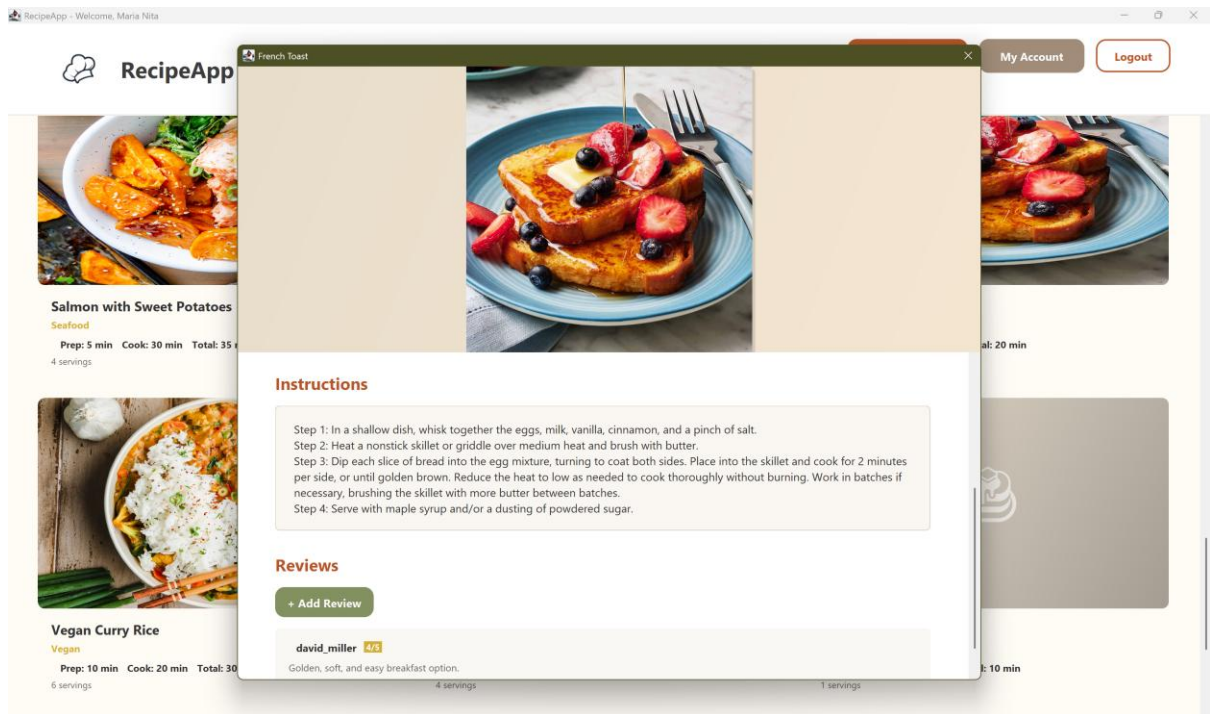


If we scroll, we can find multiple recipes

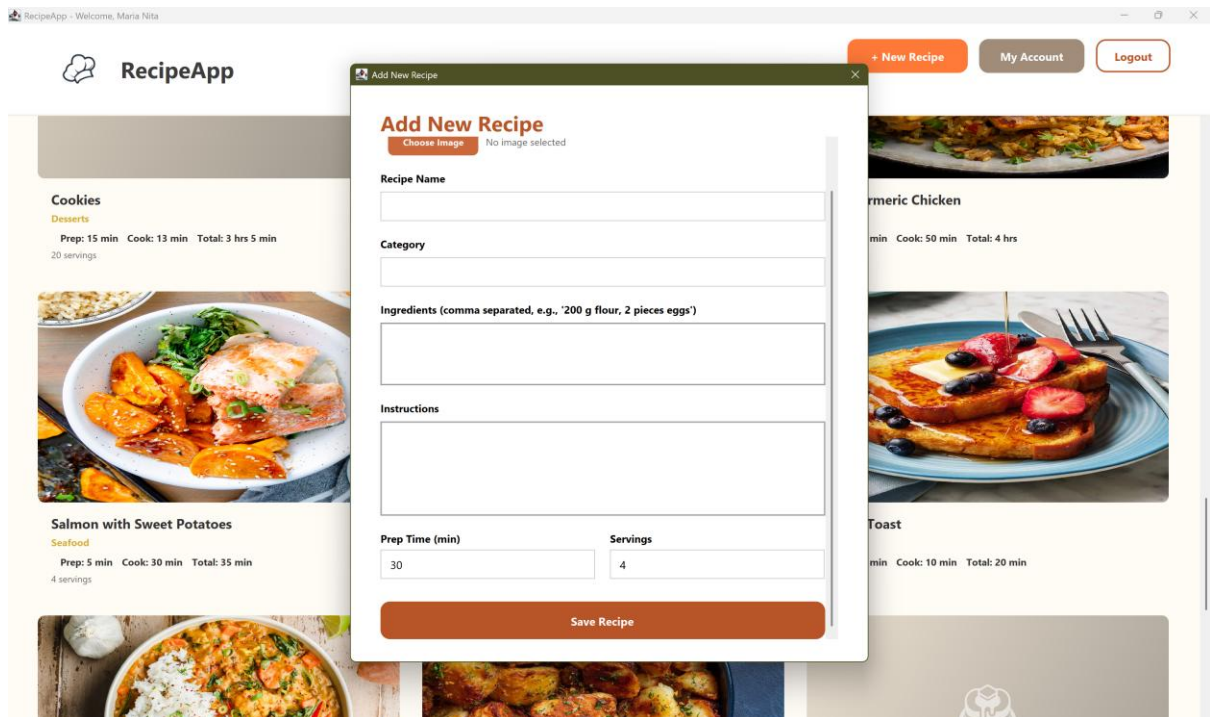


When you click on a recipe, a detailed window opens to show the full ingredients list, step-by-step cooking instructions, and reviews. This screen also allows users to rate the dish or edit the recipe if you created it.

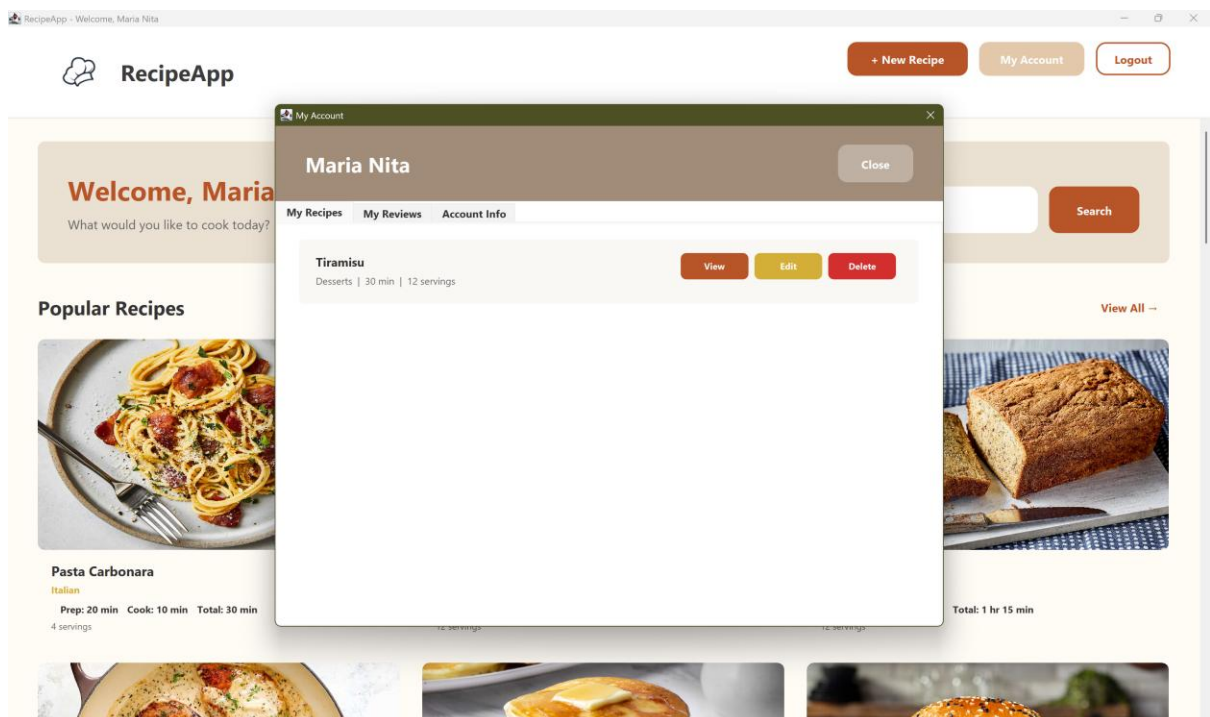




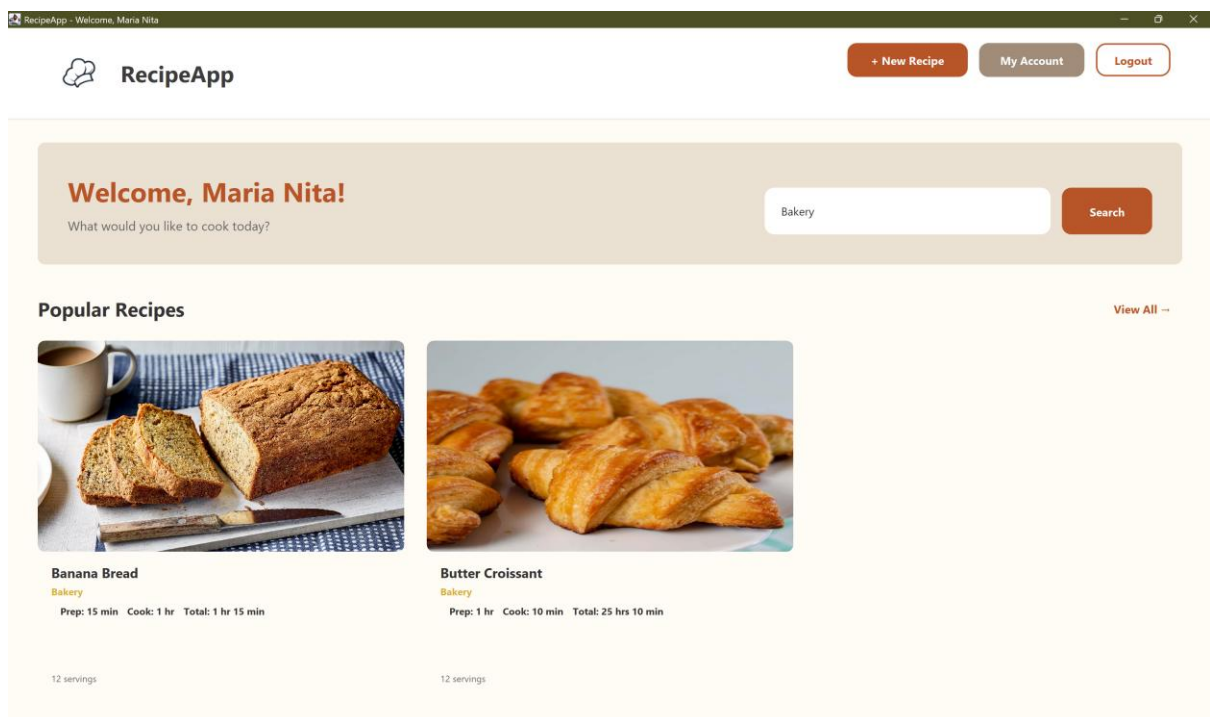
If we want to add a new recipe, we press the +New Recipe button



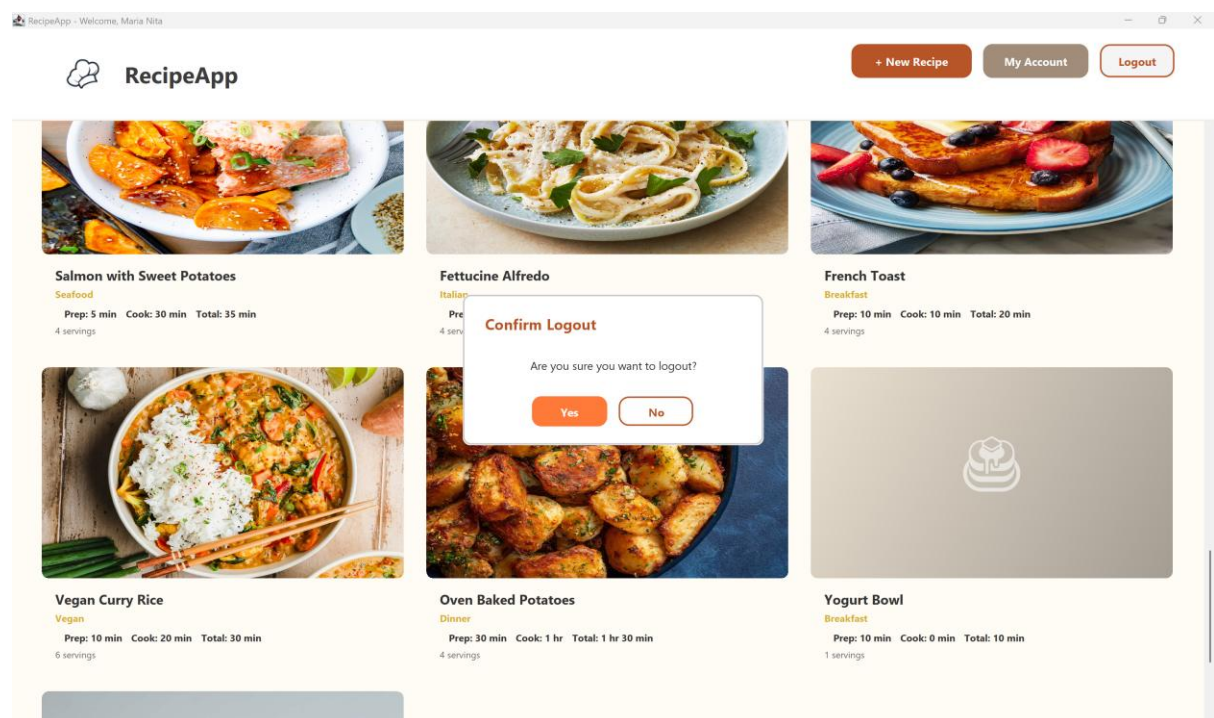
In My Account Panel we can see the recipes that the user made and the reviews he gave, as well as the account info.



The Search button allows users to quickly find recipes by typing a name or category. It instantly filters the list to show only the matching dishes, making it easy to find exactly what you want to cook.

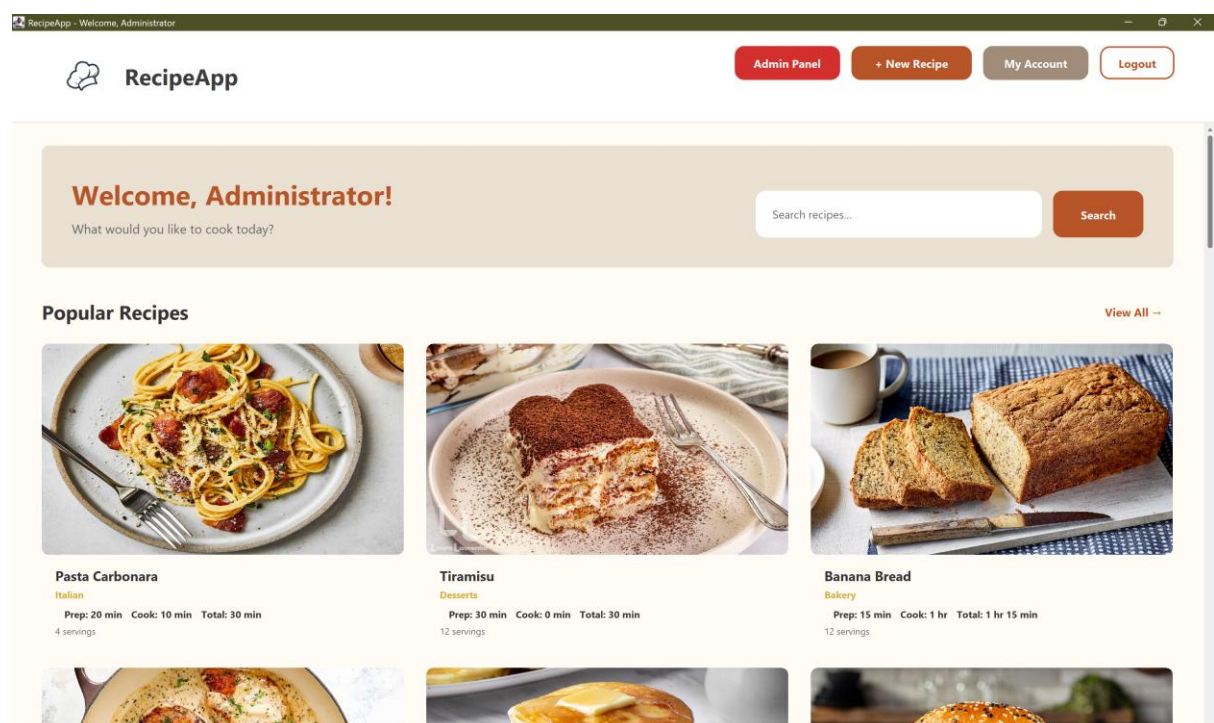


If we want to logout, we just press the Logout button and go back to the Login page.

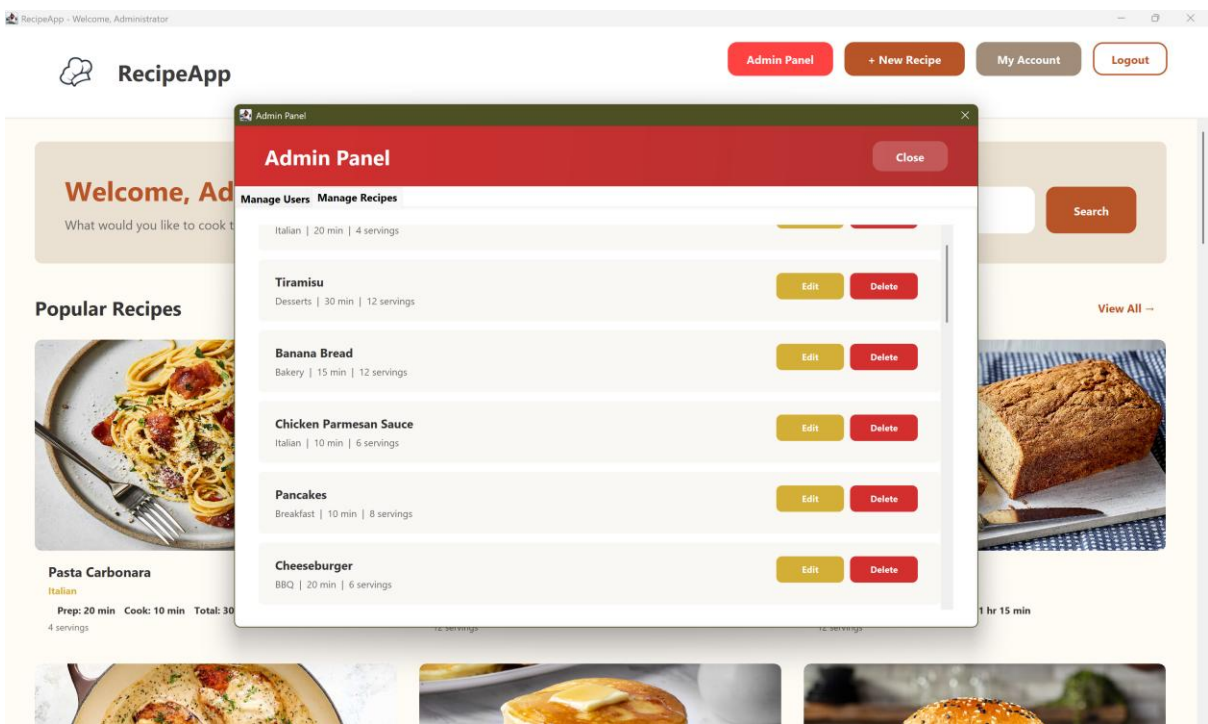
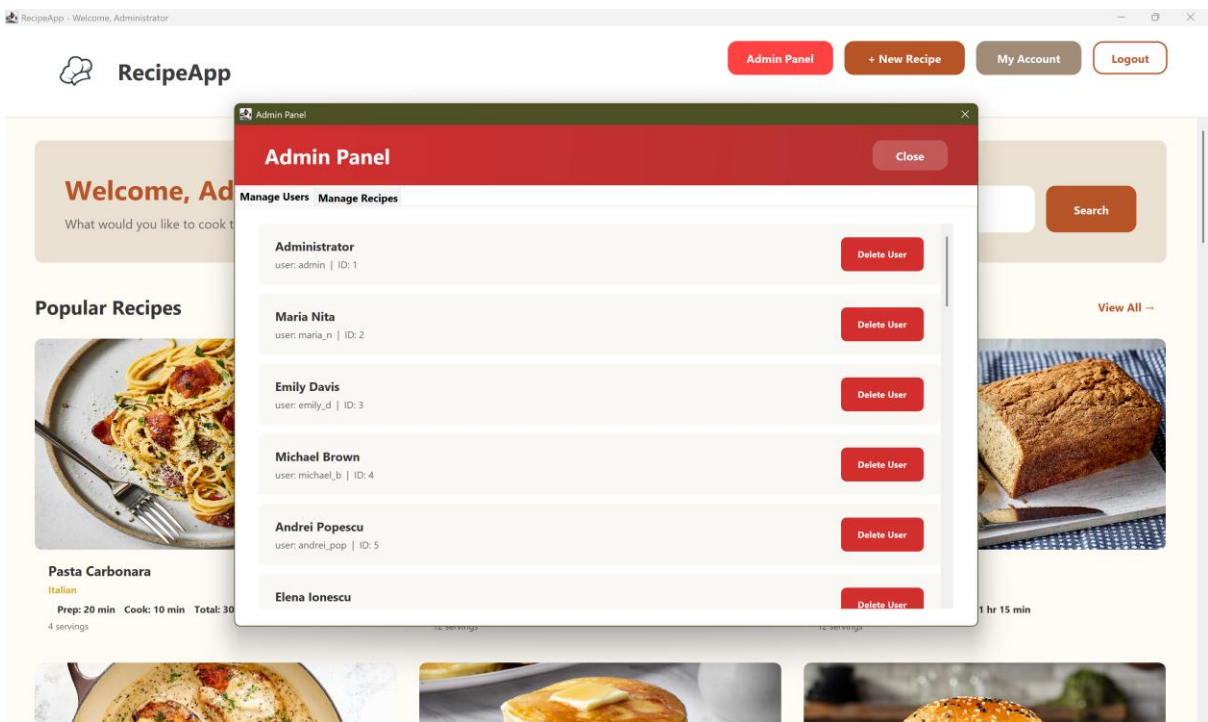


Admin page:

While normal users can only manage their own content, the Admin has full control to moderate the platform. They can access a special Admin Panel to delete any user account or remove inappropriate recipes or reviews to keep the community safe.



By pressing the Admin Panel button, the following page opens.



8. Testing

To ensure the application works correctly, we included basic unit tests (UserTest.java, ReviewTest.java). Running these tests helps catch bugs automatically before the application is run by a real user.

User Tests:

- Verifies that regular users are correctly assigned the "Home Cook" role.
- Checks that the isAdmin() function correctly identifies administrators.
- Ensures that the password checking logic accurately matches input passwords with stored ones.

Review Test:

- **Rating Validation:** Ensures that review ratings are stored correctly (e.g., verifying that a 5-star rating is retrieved as "5/5").
- **Data Integrity:** Checks that a review object correctly holds and returns the user's comment, the recipe name, and the timestamp.
- **Formatting:** Verifies that the getStarRating() helper method returns the string in the correct format expected by the UI.

9. Conclusion:

RecipeApp is a simple and useful tool for anyone who loves to cook. It allows users to save their favourite recipes, find new ideas, and share their thoughts with others, all in one easy-to-use desktop application.

By building this project, we showed how important Object-Oriented Programming (OOP) is for software development. Using concepts like Inheritance and Encapsulation helped us keep the code organized and secure. In the end, RecipeApp successfully provides a clean and reliable way for home cooks to manage their digital kitchen.